PRACTICAL 1

AIM: Project Definition and objective of the specified module and Perform Requirement Engineering Process for a **QR-Based Dine-In Ordering Platform**.

Introduction

Manual dine-in service workflows in restaurants and hotel dining halls often introduce operational bottlenecks, including delayed order-taking, mis-heard or mis-relayed items, and slow bill settlement. The **QR-Based Dine-In Ordering Platform, Dinora,** addresses these bottlenecks by serving as a browser-based table-ordering ecosystem built to remove the waits guests experience between sitting down and being served. By centralizing menu browsing, live table sessions, and order-status tracking, this application optimizes floor throughput for staff while granting guests frictionless self-service ordering and transparent order tracking

Objectives

Primary Objective

- Develop a robust, browser-based web application platform to automate, record, and optimize real-time table ordering, live order tracking, and payment settlement for dine-in restaurants and hotels.

Secondary Objectives

- Automate the order-taking pipeline across the guest's visit to systematically capture table orders alongside real-time menu availability.
- Minimize human error dependencies by removing verbal order relay between guest, server, and kitchen counter.
- Implement an intuitive self-service ordering flow to optimize guest journeys via a QR-triggered menu and transparent, live order status tracking.
- Provide restaurant owners with a real-time counter dashboard of incoming orders, sorted by table, to optimize floor staffing and service speed.

Project Definition

Scope

Dinora delivers a comprehensive web application designed to govern the dine-in ordering lifecycle through a QR-triggered, session-based architecture. The scope encompasses:

- **Live Table Session Architecture**: Provisioning a QR-triggered, session-based ordering flow — opening straight in the guest's browser with no login or install — that keeps every order for a table tied to one running session from first order to final payment.
- **Menu & Order Engine**: Maintaining a browsable digital menu with photos, dietary tags, and descriptions, while running deterministic logic to handle order placement, add-on requests, and instant availability updates.
- **Counter Dashboard Framework**: Facilitating a real-time counter queue that surfaces incoming orders tagged with table number and items, moving each ticket through New, Preparing, Served, and Paid & Closed states.
- **Owner & Payment Systems**: Structuring settlement routines to close out a table's session via cash, UPI, or card, and to update menu items and prices live across every table.

Modules

1. **QR Table Menu Module**: Governs the QR scan-to-menu flow, opening the full menu instantly in any phone browser with no app download or login required.
2. **Live Table Session Module**: Implements the shared ordering session for a table, keeping every order — including later rounds — tied to one running tab until payment.
3. **Multi-Guest Ordering Module**: Controls simultaneous ordering from multiple guests at the same table, letting everyone add to the order from their own phone.
4. **Order Ticket & Status Module**: Manages ticket creation at the counter, handles order-status transitions (New, Preparing, Served), and shifts records across the session lifecycle.
5. **Add-on & Availability Module**: Handles special requests noted directly on the ticket (extra items, no onion, less spicy) and lets staff mark dishes unavailable the instant they run out.
6. **Counter Dashboard Module**: Executes real-time queue monitoring for incoming orders sorted by table, and flags tables that need attention.
7. **Payment & Session Closure Module**: Collects cash, UPI, or card settlement and closes out the table's session once payment is confirmed.

Requirement Engineering Process

Stakeholder Identification

- **Primary Stakeholders**: Restaurant and hotel floor staff (who monitor the counter dashboard and serve incoming orders), Guests (who scan, browse, order, and pay from the table), and Restaurant Owners (responsible for menu updates and floor staffing decisions).
- **Secondary Stakeholders**: Hotel & Restaurant Management (who review service-time and turnover trends) and Platform Developers (responsible for application maintenance and system stability).

Requirement Elicitation

- **Interviews**: Structured interviews are conducted with restaurant floor staff to map out precise manual pain points regarding order relay, mis-heard items, and bill settlement delays.
- **Surveys**: Digital questionnaires are distributed to regular dine-in guests to capture menu-browsing preferences, ordering-flow demands, and waiting-time friction zones.
- **Observation**: Direct observation of a busy dining floor's order-taking and billing cycle is executed to identify systemic bottlenecks in the current waiter-dependent workflow.

Requirement Analysis

- **Functional Requirements**:
    - The system must open a live table session the instant a guest scans the table's QR code, with no login or install required.
    - The system must load the full menu with photos, dietary tags, and descriptions, and reflect availability changes instantly.
    - The system must allow multiple guests at the same table to add to a single shared order from their own phones.
    - The system must create an order ticket that reaches the counter dashboard instantly, tagged with table number and requested items.
    - The system must track order status through New, Preparing, Served, and Paid & Closed states, visible to both guest and counter.

- **Non-Functional Requirements**:
    - **Performance**: The system must achieve rapid execution, with the menu and order ticket loading in the guest's browser without a noticeable delay.
    - **Scalability**: The application architecture must easily sustain concurrent table sessions across a full dining floor during peak hours.
    - **Reliability**: The system must keep a table's session accurately tied to that table until payment closes it out, with no order loss on repeat scans.
    - **Availability**: The platform infrastructure must operate continuously through service hours, so a guest is never left unable to order.

Requirement Specification

- **Documentation**: An official Software Requirements Specification (SRS) document is structured to capture all functional and non-functional engineering limits.
- **Use Case Diagrams**: Unified Modeling Language (UML) use case diagrams are developed to map guest, counter-staff, and owner workflows across the ordering session.
- **User Stories**: Granular agile user stories are written to trace guest goals — scan, browse, order, track, and pay — across each screen of the ordering flow.

Requirement Validation

- **Reviews**: Regular technical walk-throughs and review meetings are held with restaurant stakeholders to check requirement clarity and structural completeness.
- **Prototyping**: Interactive wireframes and rapid UI mockups of the QR menu and counter dashboard are built to validate ordering-flow interactions with floor teams.
- **Testing**: Comprehensive requirements trace testing is executed to confirm that every planned ordering-flow feature corresponds directly to an engineering deliverable.

Lab Activities

Activity 1: Project Definition

- **Task**: Articulate the scope, objectives, and domain boundaries of the QR-based dine-in ordering platform, Dinora.
- **Deliverable**: A comprehensive project definition charter detailing operational limits and target user ecosystems (hotels and fine casual dining).

Activity 2: Stakeholder Identification

- **Task**: Catalog and group all individuals interacting with the dine-in ordering environment to define access needs.
- **Deliverable**: A stakeholder analysis matrix defining guest, floor-staff, and owner access needs.

Activity 3: Requirement Elicitation

- **Task**: Execute dynamic stakeholder interviews, surveys, and workflow observations on an operational dining floor.
- **Deliverable**: A raw requirement elicitation log capturing ordering-flow pain points, structural needs, and session conditions.

Activity 4: Requirement Analysis

- **Task**: Dissect, categorize, and prioritize gathered ordering-flow features into explicit functional and non-functional requirements.
- **Deliverable**: An analytical requirement categorization report detailing dependencies, parameters, and constraints.

Activity 5: Requirement Specification

- **Task**: Convert analyzed requirement statements into formal engineering models by authoring an SRS document accompanied by use case diagrams.
- **Deliverable**: A formally structured, baseline Software Requirements Specification (SRS) document.

Activity 6: Requirement Validation

- **Task**: Validate the specified boundaries with primary stakeholders using UI prototypes, structural reviews, and interface test cases.
- **Deliverable**: A signed requirement validation audit report confirming alignment across guest and floor-staff needs.

Conclusion

This initial engineering phase establishes a structured approach to scoping, defining, and formalizing the engineering foundation for Dinora. By systematically translating practical dine-in ordering workflows, live table-session states, and payment policies into verifiable technical requirements, a clear blueprint is created for subsequent software layers. Following these steps ensures the application remains securely anchored to stakeholder needs,

minimizing development deviations during architecture design, session-state design, and final interface implementation.

PRACTICAL 2

AIM: Identify Suitable Design and Implementation models from the different software engineering models for a **QR-Based Dine-In Ordering Platform (Dinora)**.

Objectives

- Understand the explicit structural, functional, and operational requirements governing the Dinora ordering platform.
- Explore and analyze diverse software development lifecycles (including Waterfall, Agile, Spiral, and the V-Model) to ascertain their individual trade-offs.
- Identify and justify the selection of the most adaptive, scalable software engineering model specifically for implementing a QR-triggered, session-based dine-in ordering web application.
- Architect a structural prototype blueprint covering use case components, live-session data models, and counter-dashboard interface boundaries.
- Establish a systematic testing and evaluation plan to verify implementation functionality against core ordering-flow rules.

Prerequisites

- Software Engineering Principles: Foundational understanding of the Software Development Lifecycle (SDLC), iterative release methods, and agile framework mechanics.
- Front-End Web Architecture: Core competency in browser-based session handling, responsive UI layout, and mobile-first interaction design.
- Data & Session Management: Practical experience with structuring live order data, table-session state, and real-time status updates.
- Tooling Proficiency: Familiarity with standard design and prototyping tools, version control protocols via Git, and UI wireframing clients.

Lab Setup

- **Software Environment**:
    - Delivery Format: Browser-based web app, QR-triggered, no install required (native app planned next).

- Primary Deployment: acenetstudios.vercel.app hosting for the current web app prototype.
- Interface Scope: QR table menu, live table sessions, and a counter dashboard for incoming orders.
- Target Venues: Hotels, fine casual dining, and quick-casual dining rooms.
- Payment Channels: Cash, UPI, and card, closed out within the same table session.

- **Hardware Specification**: Standard development computer station for the web app, alongside a counter-side device (tablet or desktop) to run the dashboard, and any guest smartphone with a camera and browser to scan and order.
- **Data Context**: Sample menu data simulating dish entries, dietary tags, table sessions, and order-status transitions across New, Preparing, Served, and Paid & Closed.

Activity 1: Requirement Analysis

- **Objective**: Define the boundaries of the system by transforming the raw service rules of a dine-in floor into rigid functional items.
- **Tasks**:
  - Detail the system's core purposes, prioritizing QR-triggered session start, menu browsing, order placement, and payment settlement.
  - Enumerate screen-specific components, tracking the guest ordering flow, the counter dashboard, and status-tracking views.
  - Group stakeholders into access tiers to map out guest, floor-staff, and owner privileges.
- **Outcome**: A detailed, verified requirement specification document aligned with dine-in service workflows.

Activity 2: Exploration of Software Engineering Models

- **Objective**: Analyze the structural behavior of industry-standard development paradigms against the specific demands of the project.
- **Tasks**:
  - Study the core mechanics of the sequential Waterfall process, iterative Agile sprints, risk-driven Spiral strategies, and verification-heavy V-Models.

- Evaluate the advantages and disadvantages of each individual framework when applied to web application patterns.
- Grade every paradigm based on variable parameters such as timeline flexibility, client input iterations, and architectural complexity.

- **Outcome**: A detailed comparative evaluation matrix mapping out the capabilities of software engineering lifecycles.

Software Engineering Model	Pros (Context of Dinora)	Cons (Context of Dinora)	Flexibility	Stakeholder Input
Waterfall	Clear initial structures; straightforward documentation maps.	High failure vulnerability; unable to handle changes mid-stream.	Rigid / Low.	Only at start / end.
Agile (Recommended)	Iterative feature delivery; seamless integration of future enhancements (e.g., native app, owner analytics).	Demands constant team communication; requires rigid sprint discipline.	Highly Adaptive.	Continuous per iteration.
Spiral	Deep risk analysis; excellent for intricate payment-settlement modules.	Highly expensive execution; over-complicated for mid-sized web stacks.	Moderate.	At every phase review.
V-Model	Strict validation links; test scenarios map directly to requirements.	Lacks early prototyping paths; making alterations late is highly expensive.	Rigid.	Only during alignment.

Activity 3: Selection of the Suitable Model

- **Objective**: Formally declare and defend the optimum execution framework to guide the system's development lifecycle.
- **Tasks**:
  - Justify utilizing an **Agile (Scrum/Kanban)** framework by highlighting the phased nature of the application (web app now, native app next, owner analytics later).
  - Map out the integration path for rolling out incremental phases, ensuring the core QR-ordering flow releases early while advanced extensions launch later.

- Plan feedback check loops at the conclusion of every feature delivery step to verify UI layout and ordering-flow controls.

- **Outcome**: An official model selection report containing engineering justifications for the selected lifecycle.

Activity 4: System Design

- **Objective**: Construct the structural, data, and behavioral layout blueprints defining the platform's guest and counter-side environments.
- **Tasks**:
  - Create comprehensive Use Case diagrams separating the roles of Guests and Counter Staff against core actions like ordering or closing a session.
  - Design session and order data models specifying how a table's live session links its orders, statuses, and payment state.
  - Draft high-level data handling flows mapping a guest's QR scan through menu load, order placement, and counter-ticket delivery.
- **Outcome**: System design documents, session-state maps, and logical system flowcharts.

Activity 5: Implementation

- **Objective**: Translate architectural design diagrams into executable, production-ready front-end software components.
- **Tasks**:
  - Initialize the workspace environment, setting up the browser-based ordering flow alongside the counter dashboard UI scaffolding.
  - Program core session-handling logic by defining state models for tables, orders, and payment status.
  - Integrate functional UI components including the QR-triggered menu, order-status tracker, and counter ticket queue.
- **Outcome**: A fully functional, responsive web app prototype of Dinora.

Activity 6: Testing and Evaluation

- **Objective**: Verify system performance metrics and check for ordering-flow discrepancies across session states.

- **Tasks**:
    - Execute strict testing over order-ticket logic, verifying that a second round of orders lands on the same running session rather than a fresh one.
    - Conduct verification on session handling to confirm a closed table cannot silently receive new orders after payment.
    - Measure baseline performance to confirm the menu and order flow load quickly on a guest's phone browser.
- **Outcome**: Comprehensive verification test reports detailing system performance metrics and ordering-flow compliance validation.

Activity 7: Documentation and Reporting

- **Objective**: Package engineering files, lifecycle choices, and testing summaries into a structured practical lab portfolio.
- **Tasks**:
    - Author a detailed technical summary document detailing development execution metrics and system build achievements.
    - Embed structural engineering schematics, session-state examples, and UI screen layout captures.
    - Compile an analytical retrospective breaking down encountered session-handling errors, environment configurations, and reliability optimization metrics.
- **Outcome**: A complete, peer-reviewed engineering lab report portfolio.

Expected Outcomes

Upon successful completion of this practical, students will achieve the following deliverables:

- **SDLC Evaluation Matrix**: A comprehensive comparative document that evaluates traditional and modern software engineering lifecycles against the architecture of a real-time ordering web application.
- **Model Selection Justification Report**: A formalized defense detailing why the Agile framework aligns best with the phased rollout of the system (web app, native app, owner analytics).

- **Functional System Architecture Blueprint**: A conceptual architectural layout mapping out the guest-to-counter request pipeline from QR scan down through order-ticket delivery.
- **Ordering Flow Mockups**: High-fidelity wireframe interfaces displaying the QR menu, live order tracking, and the counter dashboard built using responsive layouts.

Assessment Criteria

The practical submission will be graded rigorously based on the following specific engineering indicators:

Assessment Component	Weight	Grading Parameters & Quality Indicators
Requirement Analysis & Scope	**20%**	Clear classification of functional user journeys (Guest vs. Counter-Staff workflows) and realistic scoping of non-functional operational boundaries.
Lifecycle Evaluation Quality	**25%**	Analytical depth in contrasting structural patterns (Waterfall, Spiral, V-Model, Agile) and the accuracy of matching Agile sprints to iterative release phases.
System Blueprint Architecture	**25%**	Technical accuracy of conceptual data flow mappings, live-session state representations, and counter-dashboard routing steps.
Prototype Presentation & Design	**15%**	Logical arrangement of UI control structures, responsive design consistency, and clean separation of guest and counter layout states.
Technical Report & Documentation	**15%**	Professional writing style, structured formatting according to lab manual regulations, and precise usage of core ordering-platform engineering terminology.

PRACTICAL 3

AIM

Prepare Software Requirement Specification (SRS) for the selected module.

Objectives

1. Understand the functional and non-functional requirements of the system.
2. Identify the key modules and their interactions.
3. Document the requirements in a structured SRS format.
4. Ensure the SRS aligns with the goals of Dinora, the QR-based dine-in ordering platform.

Prerequisites

1. Basic understanding of dine-in restaurant service flow and its significance in guest experience.
2. Familiarity with Software Requirement Specification (SRS) documentation.
3. Knowledge of browser-based session handling and its application in ordering systems.

Tools and Resources

1. Software Tools: Microsoft Word, Google Docs, or any SRS template tool.
2. Reference Materials: IEEE SRS template, dine-in ordering case studies, and QR-ordering UX literature.
3. Hardware: Computer with internet access.

Lab Activity

Step 1: Understand the System

1. Overview: Dinora is designed to let guests scan a QR code at their table, browse the menu, and order for everyone in one shared session. It integrates a live counter dashboard to route orders instantly.
2. Key Modules:
    - **QR Table Menu Module:** Collects orders from guests via the QR-triggered menu.
    - **Live Table Session Module:** Keeps every order for a table tied to one running session.

- **Counter Dashboard Module:** Surfaces incoming tickets to the counter, sorted by table.
- **Order Status Module:** Provides a dashboard for guests and counter staff.
- **Payment Module:** Closes out a table session on cash, UPI, or card settlement.

Step 2: Select a Module

Choose one module from the system (e.g., Live Table Session Module) for which you will prepare the SRS.

Step 3: Gather Requirements

1. Functional Requirements:
   - Define the inputs, processes, and outputs of the selected module.
   - Example for Live Table Session Module:
     - **Input:** QR scan event and table identifier.
     - **Process:** Open a session, attach subsequent orders to the same session until payment.
     - **Output:** A running order ticket tied to the table, visible to guest and counter.
2. Non-Functional Requirements:
   - **Performance:** The menu and session should load in the guest's browser without a noticeable delay.
   - **Scalability:** The system should handle concurrent table sessions across a full dining floor.
   - **Security:** Ensure order and payment data privacy and reliable session handling.
3. User Requirements:
   - Define the needs of end-users (guests, counter staff, owners).
   - **Example:** Counter staff should receive new orders in an easy-to-read ticket format.

Step 4: Document the SRS

Use the IEEE SRS template or a similar structure to document the requirements. Include the following sections:

4. Introduction:
    - Purpose of the SRS.
    - Scope of the selected module.
    - Definitions and acronyms.
5. Overall Description:
    - Module functionality.
    - User characteristics.
    - Constraints and assumptions.
6. Specific Requirements:
    - Functional requirements.
    - Non-functional requirements.
    - Interface requirements.
7. Appendices:
    - Glossary, references, and additional information.

Step 5: Review and Validate

8. Cross-check the SRS with the system objectives.
9. Ensure clarity, completeness, and consistency.
10. Share the SRS with peers or instructors for feedback.

Expected Outcome

A well-structured Software Requirement Specification (SRS) document for the selected Dinora module.

PRACTICAL 4

AIM

Develop Software project management planning (SPMP) for the specified module.

Objectives

1. Understand the requirements of Dinora, the QR-based dine-in ordering platform.
2. Define the scope, deliverables, and milestones of the project.
3. Identify resources, roles, and responsibilities for the project.
4. Develop a project schedule and timeline.
5. Plan risk management, quality assurance, and communication strategies.
6. Create a comprehensive SPMP document.

Prerequisites

1. Basic understanding of software project management.
2. Familiarity with browser-based ordering flows and dine-in service operations.
3. Knowledge of tools like Trello, Jira, or Asana for project planning.
4. Understanding of QR-triggered, session-based ordering concepts.

Tools and Resources

1. Project management tools: Trello, Jira, Asana, or Microsoft Project.
2. Documentation tools: Microsoft Word, Google Docs, or LaTeX.
3. Front-end frameworks: React or similar, for reference.
4. Sample SPMP templates.

Lab Procedure

Step 1: Project Overview

1. Define the Problem Statement:
    - Develop a QR-based dine-in ordering platform to remove the waits guests experience between sitting down and being served.
    - The system should include features like QR menu access, live table sessions, counter dashboard, and flexible payments.
2. Scope of the Project:
    - **Identify the modules:** QR Table Menu, Live Table Session, Counter Dashboard, Order Status Tracking, Payments.

- Define the boundaries of the project (web app now, native app next).

3. Stakeholders:
   - **Identify stakeholders:** Guests, floor staff, restaurant owners, and developers.

Step 2: Project Deliverables and Milestones

4. Deliverables:
   - SPMP document.
   - QR-triggered menu and ordering flow.
   - Counter dashboard prototype.
   - Final deployed web app.
5. Milestones:
   - Requirement gathering and analysis (Week 1-2).
   - Design and prototyping (Week 3-4).
   - Ordering flow and session-handling development (Week 5-6).
   - Integration and testing (Week 7-8).
   - Deployment and documentation (Week 9-10).

Step 3: Resource Planning

6. Human Resources:
   - **Project Manager:** Oversees the project.
   - **Front-End Developer:** Builds the ordering flow and counter dashboard.
   - **UI/UX Designer:** Designs the guest and counter interfaces.
   - **QA Engineer:** Ensures quality and testing.
   - **Restaurant Consultant:** Provides domain expertise on floor operations.
7. Hardware and Software Resources:
   - Computers with front-end frameworks installed.
   - Cloud hosting for deployment (e.g., Vercel).
   - Mobile devices for testing the QR-scan flow.

Step 4: Project Schedule

8. Use a project management tool to create a Gantt chart.
9. Define tasks and subtasks for each milestone.
10. Allocate time and resources for each task.
11. Identify dependencies between tasks.

Step 5: Risk Management

1. Identify Risks:
   - Guest data and payment privacy concerns.
   - Delays in counter-dashboard integration.
   - Session-handling issues across simultaneous table orders.
2. Mitigation Strategies:
   - Implement secure payment handling and data protection practices.
   - Allocate buffer time in the schedule.
   - Conduct regular integration testing across concurrent sessions.

Step 6: Quality Assurance

3. Define quality standards for each deliverable.
4. Plan testing phases: Unit testing, integration testing, and user acceptance testing.
5. Assign QA tasks to the QA engineer.

Step 7: Communication Plan

1. Define communication channels (e.g., email, Slack, meetings).
2. Schedule regular team meetings and progress reviews.
3. Assign a communication coordinator.

Step 8: Documentation

1. Create the SPMP document with the following sections:
   - Introduction (Purpose, Scope, Objectives).
   - Project Organization (Team Structure, Roles).
   - Managerial Process (Schedule, Risk Management, Quality Assurance).
   - Technical Process (Tools, Frameworks, Development Methodology).

- Appendices (Glossary, References).

Expected Outcome

2. A comprehensive SPMP document for Dinora.
3. A clear project roadmap with defined milestones, deliverables, and timelines.
4. A risk management and quality assurance plan.

Assessment

1. Evaluate the completeness and clarity of the SPMP document.
2. Assess the feasibility of the project schedule and resource allocation.
3. Review the risk management and quality assurance strategies.